

Master TypeScript

CONTENTS

02	Core Types	04	Generics
03	Functions & Inference	07	Working with APIs
04	Objects, Interfaces & Types	08	TypeScript + React
05	Unions, Intersections & Narrowing	09	Utility Types
10	Errors & Debugging		

02 Core Types — Keep It Tight

●●●PRIMITIVES.TS

```
// The basics — you already know these concepts
let name: string = "Ankur";
let age: number = 25;
let isActive: boolean = true;

// Array — two ways, same thing
let skills: string[] = ["react", "ts"];
let scores: Array<number> = [90, 85];

// Tuple — fixed length, fixed types, fixed order
let point: [number, number] = [10, 20];
let user: [string, number] = ["Sherry", 30]; // name, age
```

**any vs unknown**
any turns off TypeScript completely. unknown forces you to check before using. Always prefer unknown over any.

**void vs never**
void = function returns nothing. never = function literally never finishes (throws or infinite loops).

●●●ANY-VS-UNKNOWN.TS

```
// ❌ any — avoid unless migrating JS
let danger: any = "hello";
danger.toUpperCase(); // no error — but what if it was a number?
danger.nonExistentMethod(); // still no error, scary.

// ✅ unknown — safe alternative
let safe: unknown = "hello";
// safe.toUpperCase(); // ERROR! must check type first
if (typeof safe === "string") {
  safe.toUpperCase(); // ✅ now it's fine
}

// void and never
function logMessage(msg: string): void {
  console.log(msg); // returns nothing
}

function crash(error: string): never {
  throw new Error(error); // never returns
}
```

03 Functions + Type Inference

This is where real learning starts. Understand this and you're ahead of 90% of devs.

●●●FUNCTIONS.TS

```
// Basic function typing
function add(a: number, b: number): number {
  return a + b;
}

// Optional param (?) and default param
function greet(name: string, greeting: string): string {
  return `${greeting ?? "Hello"}, ${name}`;
}

function createUser(name: string, role: string = "viewer") {
  return { name, role };
}

// Return type inference — TS figures it out
function multiply(a: number, b: number) {
  return a * b; // Inferred as number ✅
}

// Arrow functions
const square = (n: number): number => n * n;
```

●●●GENERICS-BASIC.TS

```
// Generic function — works with any type
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

getFirst([1, 2, 3]); // returns number
getFirst(["a", "b"]); // returns string

// Identity wrapper — another classic example
function wrap<T>(value: T): { value: T } {
  return { value };
}
```

04 Objects, Interfaces & Types

●●●OBJECTS.TS

```
// Inline object typing
function printUser(user: { name: string; age: number }) {
  console.log(user.name);
}

// Interface
interface User {
  id: number;
  name: string;
  email?: string; // optional
  readonly createdAt: Date; // can't be changed after creation
}

// Type alias — very similar to interface
type Product = {
  id: number;
  name: string;
  price: number;
};

// Extending interfaces
interface Admin extends User {
  permissions: string[];
}

// Intersection types (& combines)
type AdminProduct = User & Product;
```

FEATURE	INTERFACE	TYPE
Object shapes	✅ Yes	✅ Yes
Extend / inherit	✅ extends	✅ via &
Declaration merging	✅ Auto-merges	❌ Error
Unions / Primitives	❌ No	✅ Yes
When to use	Classes, public API shapes	Unions, aliases, utility types

→ Practical rule: Use **interface** for objects you'll extend. Use **type** for everything else. Don't overthink it.

05 Unions, Intersections & Narrowing

This is the backbone of real TS usage. What makes TypeScript actually useful in apps.

●●●UNIONS-AND-NARROWING.TS

```
// Union — one of these types
type ID = string | number;
type Status = "active" | "inactive" | "pending";

// typeof narrowing
function processId(id: ID) {
  if (typeof id === "string") {
    return id.toUpperCase(); // TS knows it's string here
  }
  return id.toFixed(2); // TS knows it's number here
}

// "in" narrowing
type Cat = { meow: () => void };
type Dog = { bark: () => void };

function makeSound(animal: Cat | Dog) {
  if ("meow" in animal) {
    animal.meow(); // safe!
  } else {
    animal.bark();
  }
}
```

●●●DISCRIMINATED-UNIONS.TS

```
// ★ Discriminated Unions — VERY IMPORTANT
// Add a "kind" / "type" field to each variant

type LoadingState = { status: "loading" };
type SuccessState = { status: "success"; data: string[] };
type ErrorState = { status: "error"; message: string };

type ApiState = LoadingState | SuccessState | ErrorState;

function render(state: ApiState) {
  switch (state.status) {
    case "loading": return "Loading...";
    case "success": return state.data; // data is available
    case "error": return state.message; // message is available
  }
}

// You see it? TS narrows the type in each case block!
```

06 Generics — Practical Only

●●●GENERICS.TS

```
// Generic function
function identity<T>(value: T): T {
  return value;
}

// Generic type
type ApiResponse<T> = {
  data: T;
  success: boolean;
  message: string;
};

// Now reuse for any shape
type UserResponse = ApiResponse<User>;
type ProductResponse = ApiResponse<Product>;

// Constraints with extends
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { name: "Sherry", age: 30 };
getProperty(user, "name"); // ✅
getProperty(user, "xyz"); // ❌ Error — "xyz" not in user
```

07 Working with APIs

●●●API-TYPING.TS

```
// 1. Define the shape of what the API returns
interface Post {
  id: number;
  title: string;
  body: string;
  userId: number;
}

// 2. Generic fetch wrapper — reuse everywhere
async function fetchData<T>(url: string): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new Error("Failed to fetch");
  return res.json() as Promise<T>;
}

// 3. Usage — fully typed!
const post = await fetchData<Post>("https://jsonplaceholder.typicode.com/posts/1");
console.log(post.title); // TS knows this is a string ✅

// 4. Handling unknown data safely
async function safeFetch(url: string): Promise<unknown> {
  const res = await fetch(url);
  return res.json();
}

const raw = await safeFetch("..."); // unknown
// Must validate before using — use zod, or manual checks
```

✅ Always define an interface for API responses. Type the data, not just the fetch call. Use a generic wrapper so you don't repeat yourself.

❌ Don't use `as any` to "fix" type errors from fetch. You lose all safety and introduce bugs silently.

08 TypeScript with React

This is where your value is. Make this a big chunk — not an afterthought.

●●●PROPS.TSX

```
// Typing Props
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary";
  disabled?: boolean;
}

const Button = ({ label, onClick, variant = "primary", disabled }: ButtonProps) => (
  <button onClick={onClick} disabled={disabled} className={variant}>
    {label}
  </button>
);

// Typing children
interface CardProps {
  title: string;
  children: React.ReactNode;
}

const Card = ({ title, children }: CardProps) => (
  <div><h2>{title}</h2><div>{children}</div>
);
```

●●●HOOKS-AND-EVENTS.TSX

```
// useState — infer or explicit
const [count, setCount] = useState<number>();
const [user, setUser] = useState<User | null>(null);

// useEffect — nothing special needed
useEffect(() => {
  const loadUser = async () => {
    const data = await fetchData<User>("/api/user");
    setUser(data);
  };
  loadUser();
}, [1]);

// Event types — very common question!
const handleClick = (e: React.MouseEvent<HTMLInputElement>) => {
  console.log(e.target.value);
};

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
};

const handleClickCurrent = (e: React.MouseEvent<HTMLButtonElement>) => {
  console.log(e.currentTarget);
};
```

●●●CUSTOM-HOOKS.TSX

```
// Custom hook with return type
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState<boolean>();
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetchData<T>(url)
      .then((data) => setData(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, [url]);

  return { data, loading, error };
}

// Usage
const [data, loading] = useFetch<Post>(`/api/posts`);
```

09 Utility Types — High Impact Only

●●●UTILITY-TYPES.TS

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
}

// Partial — all fields optional (great for update functions)
type UserUpdate = Partial<User>;
// { id?: number; name?: string; email?: string; age?: number }

function updateUser(id: number, changes: Partial<User>) { ... }

// Pick — take only what you need
type UserPreview = Pick<User, "id" | "name">;
// { id: number; name: string }

// Omit — exclude what you don't need
type PublicUser = Omit<User, "email">;
// { id: number; name: string; age: number }

// Record — key-value mapping with typed keys and values
type RoleMap = Record<"admin" | "editor" | "viewer", string[]>;
const permissions: RoleMap = {
  admin: ["read", "write", "delete"],
  editor: ["read", "write"],
  viewer: ["read"],
};
```

Partial<T>

All fields become optional. Use for update payloads, form state, PATCH requests.

Pick<T, K>

Select specific fields. Use for DTOs, API response subsets, preview cards.

Omit<T, K>

Exclude fields. Use to hide sensitive data (passwords, tokens) from public types.

Record<K, V>

Typed dictionary/map. Use for permission maps, caches, config objects.

10 Error Handling & Debugging Types

●●●READING-ERRORS.TS

```
// Common TS error — "Property X does not exist on type Y"
interface Car { brand: string; }
const car: Car = { brand: "Toyota" };
car.speed; // ❌ Property 'speed' does not exist on type 'Car'
// Fix: Add speed to interface, or use optional chaining

// "Type X is not assignable to type Y"
let score: number = 99;
score = "ninety"; // ❌ Type 'string' is not assignable to type 'number'

// Argument type mismatch
function double(n: number) { return n * 2; }
double("5"); // ❌ Argument of type 'string' is not assignable to parameter of type 'number'
```

●●●TYPE-ASSERTION.TS

```
// "as" — type assertion. Use sparingly.

// ✅ OK — you know more than TS does (e.g., DOM APIs)
const input = document.getElementById("name") as HTMLInputElement;
console.log(input.value);

// ✅ OK — casting API response you've validated
const data = response as User;

// ❌ BAD — using "as" to silence errors (lying to TypeScript)
const broken = "hello" as unknown as number;
broken.toFixed(); // crashes at runtime — TS was fooled

// Rule: "as" is for when YOU know better than TS.
// Not for when you're too lazy to fix a real type error.
```

→ How to read TS errors: Start from the bottom of the error stack. The first line describes what went wrong. The last line tells you where. Read both.

Quick Debug Trick

Hover over any variable in VS Code to see its inferred type. If it's `any`, `unknown`, or `any[]`, you need to fix something upstream.

Fixing Mismatches

Don't silence errors with `as`. Fix the type definition or the function contract. The error is telling you something bad.

ts-expect-error

Use `// $ts-ignore` instead of `// @ts-ignore`. It tells the error disappears — is self-cleaning suppression.